KasSigner + KasSee Covenants & Stealth Guide

Covenant, swap, crowdfunding, oracle, and stealth authorization workflows

1. Security boundary

Covenants and advanced workflows are constructed online in KasSee and authorized offline by KasSigner. The signer validates recognized structure and presents relevant spend context. Unknown or opaque commitments receive stronger warnings.

> Ordinary wallet-spending keys do not expose a generic caller-selected SIGN HASH operation. Covenant, oracle, and swap authority uses isolated derivation domains so advanced protocols do not turn the ordinary wallet key into a generic signing oracle.

2. Supported covenant families

Family	Purpose / boundary
Piggy Bank	Constrained savings/spend path.
Time-Locked Savings	Spending after configured time/DAA condition.
Dead Man's Switch	Primary/recovery timing policy.
Allowance	Bounded recurring/authorized spending.
Spending Limit	Amount-limited authorization.
Merkle Whitelist	Spend restricted to approved destinations.
Direct Channel / PayJoin / Commit-Reveal	Structured cooperative transaction workflows.
KIP-20 Vaults	Vault/recovery policy.
ZK Crowdfunding / Price Oracle	Proof-bound campaign/oracle workflows with explicit recovery paths.

3. Private Swap

Private Swap uses transaction-sighash-bound adaptor signatures and isolated swap keys. It is not an HTLC/hashlock replacement. KasSigner reviews the exact canonical claim transaction and participates in host-assisted anti-klepto nonce binding; completed on-chain claims are ordinary transaction signatures.

4. COVENANT SIGN

COVENANT SIGN authorizes an exact reviewed 32-byte external commitment with an isolated mnemonic-derived covenant key. Recognized schemes are recomputed/validated on-device. Opaque/custom commitments require stronger confirmation and never use the ordinary wallet-spending derivation hierarchy.

5. Oracle and crowdfunding

- Oracle signing uses a protocol-specific isolated key and canonical release-statement commitment.
- Crowdfunding flows bind campaign parameters, proof verification, bounded sweep behavior, and contributor timeout/refund semantics.
- KasSee constructs and monitors; KasSigner remains the authorization boundary.

6. Stealth payments

Stealth payments use dual-key ECDH-derived addressing. KasSee can create and scan candidate outputs; KasSigner verifies/authorizes spending through the relevant isolated key and transaction review. Never treat the online scanner result as sufficient authorization by itself.

7. Operational checklist

- Use current KasSee + KasSigner protocol formats; rebuild abandoned historical transactions rather than reopening retired signing/session parsers.
- Verify network, amount, fee, change, lock/timing conditions, participant keys, and recovery path on the hardware screen.
- For multisig/covenant change, rely on signer-proven ownership rather than labels supplied by the online companion.
- Keep protocol-specific recovery material and wallet mnemonic/passphrase backups offline.